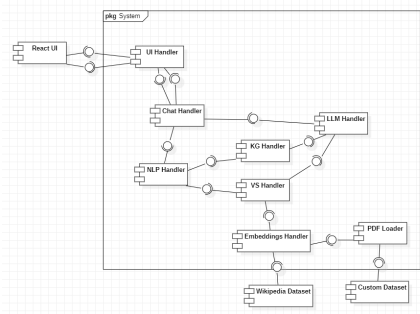


RAG Application Using Knowledge Graph and Vector Search

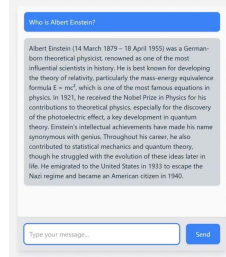
Molly Iverson Ethan Villalovoz Chandler Juego Adam Shtrikman

Washington State University

github.com/ethanvillalovoz/knowledge-graph-rag-assistant



Architecture. Hybrid retrieval pipeline.



Interface. Final response view.

Final Stats				
Component	Call 1 (ms)	Call 2 (ms)	Call 3 (ms)	Average (ms)
NLP Handler	36.40	27.40	36.40	33.40
Knowledge Graph	1498.20	1392.70	1212.30	1367.73
Vector Search	3717.20	4672.20	4011.20	4133.53
LLM	2527.00	1884.80	1827.29	2079.70
Total	7779.70	7977.60	7087.90	7615.07

Latency record. Three final runs.

Figure 1. System overview and final prototype. The application combines natural-language processing, DBpedia context, dense retrieval, and language-model synthesis. The timing panel reproduces the final three-run record for one prompt; Table 4 states its limits.

Abstract

Retrieval-augmented generation (RAG) conditions a language model on information retrieved from an external corpus. This report presents KG-RAG ASSISTANT, a senior capstone application that combines two retrieval paths: dense vector search over more than 10,000 Wikipedia articles and structured entity context from DBpedia. The system also supports a replaceable collection of PDF class notes to demonstrate retrieval over private or domain-specific material. A React interface communicates with a FastAPI backend, where spaCy processes the question, Sentence-Transformer encodes it, FAISS retrieves semantically similar text, SPARQL queries DBpedia for the first recognized entity, and an OpenAI model synthesizes the resulting context. The team packaged the frontend and backend with Docker and exercised the implementation through unit, integration, functional, performance, and client-acceptance testing. All representative cases documented in the final report passed. For the prompt “Who is Alan Turing?”, the final three recorded runs averaged 7.615 seconds end to end; because the report did not preserve hardware and full runtime metadata, this number is presented as a historical engineering measurement rather than a reproducible benchmark.

1. Introduction

Retrieval-augmented generation (RAG) combines a generative language model with information retrieved from external sources. The project described in this report was developed as a Spring 2025 senior capstone for HackerEarth. Its purpose was to build a working application that could retrieve relevant information from both unstructured text and a structured knowledge source, then use that context to produce a natural-language answer.

The application uses two retrieval paths. One performs vector search over more than 10,000 Wikipedia articles and a replaceable collection of PDF class notes. The other extracts an entity from the question and queries DBpedia. The retrieved text and DBpedia result are supplied to an OpenAI language model, and the generated response is displayed in a browser-based chat interface. The system was implemented as a React frontend and a FastAPI backend, with separate handlers for natural-language processing, vector search, knowledge-graph access, custom-data preparation, and response generation.

This manuscript presents the project’s requirements, architecture, implementation, test records, timing measurements, prototype, delivery information, limitations, and conclusions in a compact academic format. Its emphasis is the design and integration of the delivered system; it does not recast the capstone prototype as a comparative machine-

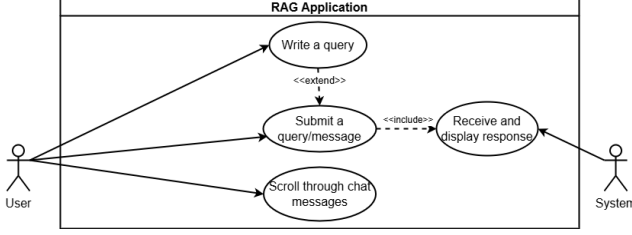


Figure 2. Original use-case diagram. The user writes and submits a query, receives a response, and can scroll through the current conversation.

learning study.

2. Project Requirements Specification

2.1. Stakeholders and scope

The primary stakeholders were HackerEarth, represented by CEO Vikas Aditya, and the Washington State University capstone instructor. The team was expected to deliver a documented, maintainable RAG application that could be demonstrated and run outside the development environment. HackerEarth did not plan to integrate the prototype into its production codebase; the artifact served as a technical demonstration and a foundation for possible future work.

The user is the system’s primary actor. Four top-level use cases organize the interaction: entering a question, submitting it, receiving a generated response, and reviewing earlier messages in the current session. Figure 2 reproduces the original UML use-case diagram, following the notation used in the project documentation [2].

2.2. Functional and quality requirements

Table 1 condenses the requirements in the final report. Priority 0 denotes essential functionality; Priority 1 denotes desirable functionality. The original non-functional requirements were partly aspirational: the project measured response time, but did not systematically validate 99% uptime, security, or degradation-free scalability.

3. System Design

3.1. Architecture

The application follows a client-server architecture (Figure 3). React manages message entry, loading state, and response display [6]. A FastAPI backend exposes endpoints for natural-language processing, DBpedia access, vector search, and generation. The original design decomposed these responsibilities into UI, chat, NLP, knowledge-graph, vector-search, embedding, PDF-loading, and LLM handlers using a conventional component-diagram notation [3]. This separation allowed team members to develop and test sub-systems independently before integration.

3.2. Data preparation

The project uses Wikipedia text and PDF class notes as retrieval sources. Raw and cleaned text are stored in Parquet files. The PDF loader extracts text from each document, converts the result to a data frame, and concatenates it with the Wikipedia data. The embedding pipeline removes missing text, optionally chunks long content, encodes the resulting rows, and stores the vectors as a NumPy array. A FAISS index preserves the mapping from a retrieved vector position back to the corresponding text row.

The final report describes a corpus of more than 10,000 Wikipedia articles. Because the project record does not preserve a release manifest that fixes the exact corpus rows, preprocessing revision, and artifact checksums used for the April demonstration, this manuscript does not infer a more precise corpus size.

3.3. Natural-language processing

The NLP endpoint applies spaCy’s English pipeline to tokenize the question, identify named entities, and detect a small set of harmful-intent keywords. If at least one entity is present, the first entity is used to construct a SPARQL query for an English DBpedia abstract. For example, “Tell me about Albert Einstein” yields an entity span for “Albert Einstein” and a query for the corresponding labeled resource.

This rule is transparent and inexpensive, but it has clear limitations. It does not disambiguate two resources with the same label, reconcile aliases, or represent relations among multiple entities. The harmful-intent detector likewise checks a small keyword list and should not be interpreted as a general moderation model. The final prompt adds more detailed refusal instructions, but prompt rules do not replace a safety evaluation.

3.4. Dense retrieval

The vector path encodes text with all-MiniLM-L6-v2, using the SentenceTransformers interface [4], and stores normalized embeddings in a FAISS IndexFlatIP index [5]. At query time, the same model encodes the question, the vector is normalized, and FAISS returns up to ten inner-product neighbors. Results below 0.5 similarity are discarded. The corresponding text rows are then loaded from the cleaned Parquet corpus.

Because both sides are normalized, inner product implements cosine similarity:

$$s(q, d) = \frac{f(q)^\top f(d)}{\|f(q)\|_2 \|f(d)\|_2}, \quad (1)$$

where $f(\cdot)$ is the SentenceTransformer encoder. The project uses a flat index, so retrieval is exact with respect to the stored vectors; observed relevance still depends on the encoder, corpus, text segmentation, threshold, and source-to-row alignment.

Table 1. Requirements preserved from the final report. “Validated evidence” distinguishes implemented or tested behavior from a design target.

Requirement	Priority	Intended behavior	Evidence documented in the report
RAG architecture	0	Retrieve external context and generate a natural-language answer.	End-to-end prototype, interface screenshots, and manual functional tests.
Vector search	0	Embed, index, and retrieve semantically related text with FAISS.	Unit cases for indexing, retrieval, and load behavior; deployed pipeline.
Knowledge graph	0	Query DBpedia and provide structured entity context.	Normal-query and no-result cases; integrated prototype.
Query response	0	Combine retrieved context and answer the submitted question.	Manual system tests and client demonstrations.
Custom dataset	1	Add PDF class notes to the retrieval corpus.	PDF-to-Parquet and embedding path in the code and prototype.
Chat window	0	Accept, submit, and display messages.	React prototype and use-case walkthrough.
Error handling	1	Surface invalid input, network, and service failures.	Alternative use-case paths and selected handler tests.
Content moderation	1	Refuse clearly harmful requests while allowing legitimate sensitive topics.	Keyword detection and generator instructions; no broad safety evaluation.
Scalability	–	Handle more than 10,000 Wikipedia articles.	Corpus-scale prototype; no controlled scaling study.
Response time	–	Return typical responses within 15 seconds.	Three-run final measurement averaged 7.615 seconds for one prompt.
Storage and maintainability	–	Persist artifacts and support modular extension.	Parquet, NumPy, and FAISS files; handler decomposition and Docker packaging.
Reliability, security, usability	–	Target availability, secure handling, and an intuitive interface.	Usability feedback and demonstration only; no uptime or security study.

3.5. Knowledge-graph context and generation

The DBpedia endpoint [1] executes the generated SPARQL 1.1 query [13] and returns JSON. The client selects the first available abstract, falling back to a no-context message when no result exists. The LLM handler then builds a prompt containing the original question, the vector-search passages, and the DBpedia text. The prompt requests a concise answer of at most 150 words, asks the model to acknowledge missing external context, and includes refusal guidance for harmful requests.

The April 2025 code uses OpenAI’s `gpt-4o-mini` through the provider’s API [8]. The retrieved passages and DBpedia abstract are supplied as context, but the interface does not attach citations to individual claims. Consequently, a generated answer may be grounded in the supplied text, use parametric model knowledge, or mix both. The application therefore supports context-augmented answering; it does not establish response faithfulness.

4. Projects and Tools Used

Table 2 summarizes the implementation stack. The backend is written in Python and the frontend in TypeScript, with Node.js supporting the frontend toolchain [9]. SPARQL serves as the query language for DBpedia. GitHub Actions runs automated tests [10], while Docker and Docker Com-

Table 2. Implementation stack and responsibility.

Component	Role
React + TypeScript	Chat interface and orchestration of backend requests.
FastAPI	HTTP API for NLP, DBpedia, vector search, and LLM response generation.
spaCy	Tokenization, entity recognition, and keyword-based intent flag.
Sentence-Transformer	Dense encoding with <code>all-MiniLM-L6-v2</code> .
FAISS	Exact inner-product search over normalized embeddings.
DBpedia	English abstract for the first detected entity.
SPARQL	
OpenAI	Context-conditioned response synthesis.
Parquet + NumPy	Text rows and embedding-matrix storage.
PyTest + GitHub Actions	Automated component and integration checks.
Docker Compose	Coordinated frontend/backend deployment.

pose package the two services [11, 12].

5. Test Case Specifications and Results

The evaluation followed a software-engineering validation strategy rather than a comparative machine-learning study.

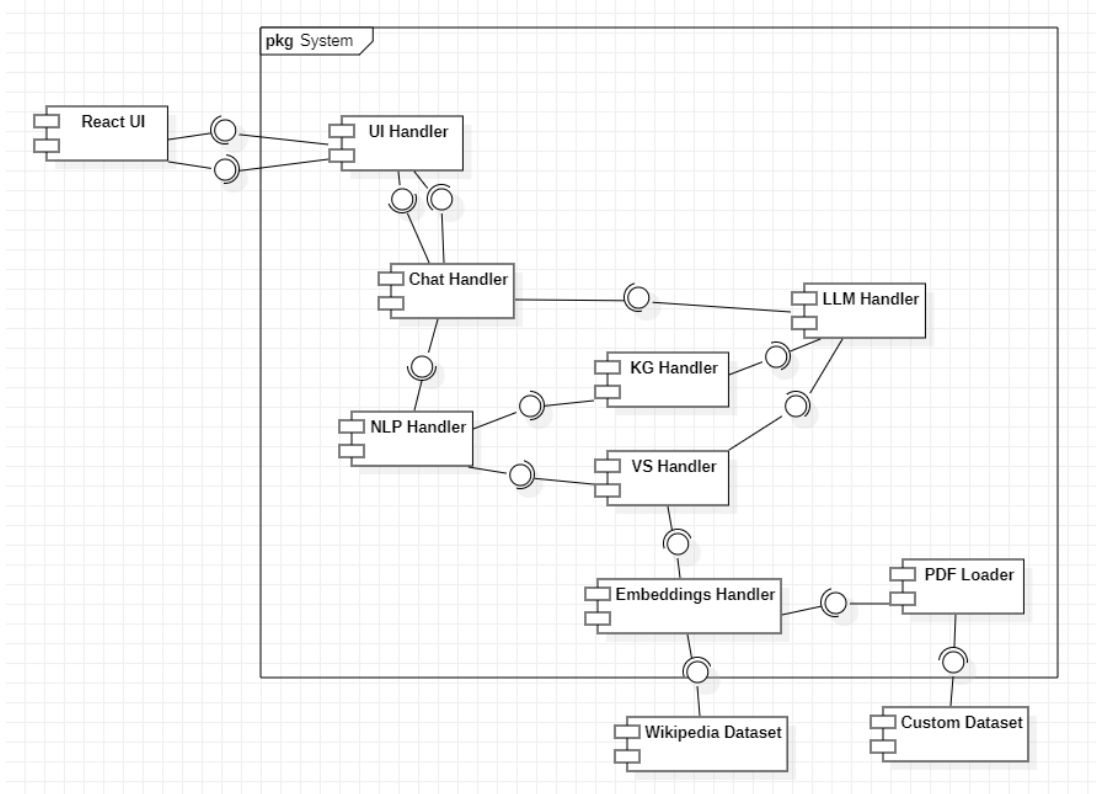


Figure 3. Component architecture from the final report. The question is processed by NLP, routed to the DBpedia and vector-search subsystems, and combined for language-model generation. The diagram documents logical responsibilities; the April 2025 client implementation invoked the DBpedia and vector endpoints sequentially.

It combined unit tests, integration tests, manual functional checks, timing measurements, and feedback during weekly client demonstrations. This distinction matters: the evidence supports implementation functionality under the tested conditions, but not a general claim that the hybrid design is more accurate than vector-only RAG.

5.1. Unit and integration testing

Unit tests isolated NLP, DBpedia, vector-search, and LLM behaviors using PyTest [7]. The documented cases covered normal and ambiguous entity extraction, empty-entity inputs, tokenization, harmful-keyword detection, DBpedia success and no-result handling, FAISS indexing and retrieval, retrieval under a large synthetic embedding set, and response-length compliance. Table 3 reproduces the representative cases from the report; all were marked as passing.

Integration testing progressively connected NLP, vector search, DBpedia, and generation. The report states that the team began with pairs of handlers and added components until the complete flow could be exercised. Manual system tests mapped core features to functional requirements. A CI screenshot in the supporting project document records 16 passing tests with four warnings, while the report’s de-

tailed table lists twelve representative cases; the difference is retained rather than silently reconciled.

5.2. Performance observations

The team measured component and total latency for the prompt “Who is Alan Turing?” across three calls at three development stages. Table 4 transcribes those measurements. In the initial record, the local LLaMA generator did not complete, so the 6.469-second total excludes a working generation stage and cannot be compared directly with the later end-to-end rows. Project notes separately report that local LLaMA attempts could require five to ten minutes on one developer machine.

After switching to OpenAI and changing the data preparation, the complete three-run average was 6.287 seconds. The final artifact averaged 7.615 seconds: 0.033 seconds for NLP, 1.368 for DBpedia, 4.134 for vector search, and 2.080 for generation. The final number satisfied the project’s 15-second target for this prompt. The increased vector-search time was attributed in the report to embedding the complete project corpus rather than the smaller data used during earlier development.

Table 3. Representative test cases reported by the team. The table records software behavior, not answer-quality accuracy.

Case	Expected behavior	Observed behavior	Result
Entity extraction: normal	Extract named entities.	Named entities extracted.	✓
Entity extraction: edge cases	Handle ambiguous and multi-word entities.	Expected entities extracted.	✓
Missing entities	Tokenize without inventing an entity.	No entity extracted.	✓
Tokenization: normal	Produce the expected word segmentation.	Output matched expectation.	✓
Tokenization: punctuation	Handle punctuation and special characters.	Output matched cleaned segmentation.	✓
Harmful-intent keyword	Flag a question containing a harmful action.	<code>is_harmful</code> returned true.	✓
DBpedia query	Execute SPARQL and return relevant data.	Endpoint returned information.	✓
DBpedia no result	Continue without raising an application error.	“No results” path returned.	✓
FAISS indexing	Index embeddings and accept a query.	Vectors indexed and searched.	✓
Vector retrieval	Return top- k document indices.	Relevant fixture indices returned.	✓
Vector-search load	Complete search over a large test set within the test’s limit.	Retrieval remained within the accepted test bound.	✓
LLM response length	Respect the configured output limit.	Response stayed within the limit.	✓

Table 4. Historical component timing averages from three calls of one prompt. “Incomplete” means the generator did not return. Hardware, variance, warm-up policy, network conditions, and software revisions were not recorded, so these values are not a reproducible benchmark.

Stage	NLP (ms)	DBpedia (ms)	Vector (ms)	LLM (ms)	Total (ms)	Interpretation
Initial	483.57	1135.17	4850.27	–	6469.30	Incomplete generator
OpenAI + data changes	240.50	1397.90	1808.53	2839.23	6286.90	Complete three-run average
Final prototype	33.40	1367.73	4133.53	2079.70	7615.07	Complete three-run average

5.3. Functional and acceptance testing

Developers manually submitted questions, inspected retrieval and generation behavior, and checked the interface’s loading and error states. Weekly HackerEarth meetings provided iterative user-acceptance feedback on the functional prototype. The client confirmed that the demonstrated system met the agreed capstone requirements. Because neither the report nor repository preserves a fixed question set, reference answers, blinded ratings, or an inter-rater protocol, statements about answer accuracy and relevance remain qualitative.

6. Final Prototype

The prototype presents a minimal chat interface. A user enters a question, the interface displays a loading state while the backend executes, and the resulting answer appears in the conversation (Figure 4). The example asks “Who is Albert Einstein?” and demonstrates the final rendered response. The same interface can query Wikipedia-derived text or the combined corpus containing class notes.

The custom-data path was included to demonstrate a broader product idea: the retrieval corpus can be replaced or extended without changing the basic interaction model. Class notes served as the project example, but the same architecture could be adapted to internal company documents or other domain collections, subject to appropriate access

control, data governance, evaluation, and licensing.

7. Product Delivery

The delivered setup exposed the React interface on port 3000 and the FastAPI service on port 8000. Users could pull the frontend and backend images described by the team, provide an OpenAI API key, and launch both services through Docker Compose. The report also documented a local-development path using Python 3.9 or later, Node.js, downloaded embedding artifacts, and separate frontend and backend processes. Code and deployment were reported complete on April 18, 2025. HackerEarth did not integrate the prototype into its production system; the application was delivered as a demonstration and foundation for possible future development.

8. Limitations and Future Work

Answer quality was not benchmarked. The team manually reviewed outputs but did not evaluate exact match, factuality, citation correctness, retrieval recall, or human preference on a fixed dataset. A publication-quality effectiveness claim would require a frozen corpus and question set, vector-only and graph-only baselines, repeated trials where generation is stochastic, and a documented human or automatic scoring protocol.

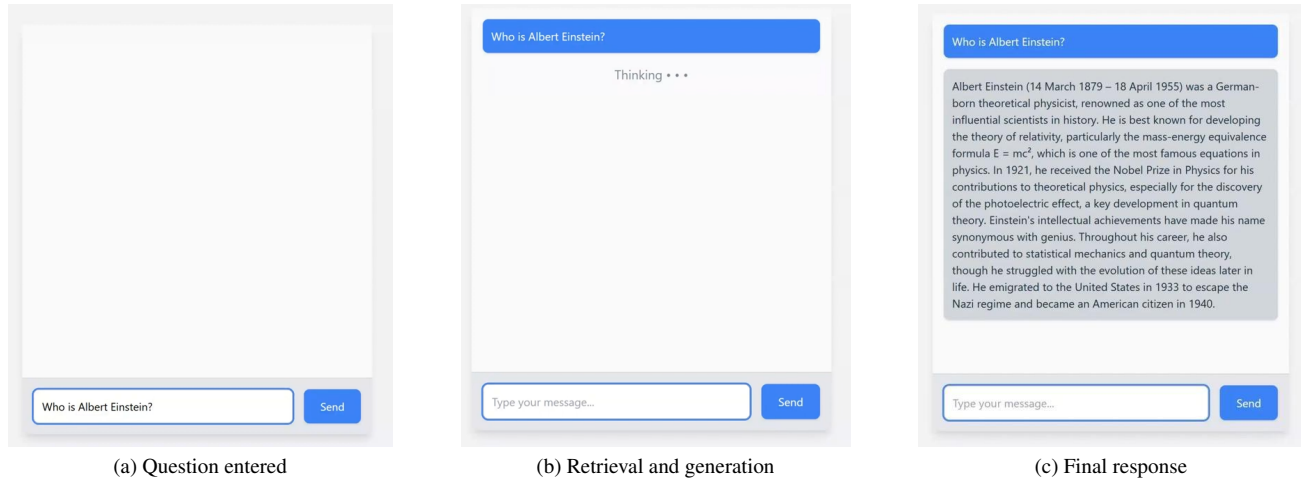


Figure 4. Original prototype sequence reproduced from the final report.

Entity processing is deliberately shallow. The system uses the first spaCy entity and an exact English label in SPARQL. Questions with multiple entities, aliases, ambiguous names, or complex relations can therefore return no context or the wrong resource. Future work could add entity linking, candidate disambiguation, relation-aware retrieval, and explicit provenance for every returned fact.

External services introduce variability. DBpedia availability and OpenAI latency affect the response path. The April artifact did not cache frequent queries, log a full reproducibility manifest, or provide an offline generator. Future deployments should record model and corpus versions, validate index-to-text alignment, cache safe repeated work, and expose source-level citations.

Several non-functional goals remain unverified. The project demonstrated a corpus exceeding 10,000 Wikipedia articles and met its latency target on one recorded prompt, but it did not measure throughput, tail latency, uptime, security, or usability with a representative user cohort. The 99% availability target should be treated as a requirement, not a measured result.

Product extensions. The original report proposes richer custom datasets, multimodal input for diagrams and charts, improved moderation, user accounts, persistent chat history, and memory across conversations. These are reasonable extensions, but each introduces new privacy, safety, storage, and evaluation requirements.

9. Conclusion

The team designed, implemented, tested, packaged, and demonstrated an end-to-end RAG application that com-

bines semantic retrieval, DBpedia context, custom documents, and language-model generation. Its main achievement is systems integration: data preparation, dense indexing, entity-based structured lookup, prompt construction, a browser interface, automated tests, and Docker delivery operate within one prototype. The historical tests support the functionality of the implemented components, and the recorded final prompt completed within the project’s response-time target. They do not establish a general accuracy advantage from knowledge-graph augmentation. By stating that boundary directly, the project provides a credible foundation for future evaluation rather than overstating what a capstone prototype can prove.

Author Contributions

Molly Iverson led planning, knowledge-graph integration, continuous integration, and deployment. Ethan Villalovoz developed SentenceTransformer embeddings, vector search, and system documentation. Chandler Juego contributed to embeddings, frontend development, vector-search optimization, and client-server integration. Adam Shtrikman developed natural-language processing, custom-data integration, and the automated testing pipeline.

Acknowledgements

The team thanks Vikas Aditya, CEO of HackerEarth, for weekly technical feedback and mentorship, and Dr. Parteek Kumar for guidance throughout the capstone development and documentation process.

References

- [1] DBpedia. About DBpedia. <https://www.dbpedia.org/about/>.

- [2] StarUML. Use Case Diagram. <https://docs.staruml.io/working-with-uml-diagrams/use-case-diagram>.
- [3] Visual Paradigm. What is Component Diagram? <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/>.
- [4] SentenceTransformers. SentenceTransformers Documentation. <https://sbert.net/>.
- [5] Facebook Research. FAISS: A library for efficient similarity search and clustering of dense vectors. <https://github.com/facebookresearch/faiss>.
- [6] React. React Reference Overview. <https://react.dev/reference/react>.
- [7] pytest. pytest documentation. <https://docs.pytest.org/en/stable/>.
- [8] OpenAI. OpenAI Developer Platform. <https://platform.openai.com/docs/overview>.
- [9] Node.js. Index: Node.js v23.11.0 Documentation. <https://nodejs.org/docs/latest/api/>.
- [10] GitHub. GitHub Actions, 2025. <https://github.com/features/actions>.
- [11] S. Ratliff. Docker: Accelerated Container Application Development. Docker, January 23, 2025. <https://www.docker.com/>.
- [12] Docker. Docker Compose. Docker Documentation, November 26, 2024. <https://docs.docker.com/compose/>.
- [13] World Wide Web Consortium. SPARQL 1.1 Query Language. <https://www.w3.org/TR/sparql11-query/>.

A. Supporting Project Documentation

This appendix preserves software-engineering material from the team’s final capstone report that supports the project record but is not required in the main narrative. It records subsystem responsibilities, the testing environment, original timing screenshots, operating instructions, project-management practices, and terminology.

A.1. Subsystem Responsibilities

UI handler. Manages the React layout, visible message state, and input events, then routes user actions to the chat logic.

Chat handler. Coordinates question submission and response display, routing each question through the processing and generation sequence.

NLP handler. Tokenizes the submitted question, extracts named entities, checks selected harmful-intent keywords, and constructs the SPARQL request used by the knowledge-graph path.

Knowledge-graph handler. Executes the SPARQL request against DBpedia and returns the structured response to the client-side orchestration path.

Vector-search handler. Encodes the question, searches the FAISS index, and maps returned vector positions to their corresponding text passages.

Embedding and PDF handlers. The embedding handler loads, cleans, optionally segments, and encodes source text. The PDF loader extracts class-note text and stores it in Parquet form for inclusion in data preparation.

LLM handler. Formats the question and retrieved context, calls the configured OpenAI model, and returns the natural-language response displayed in the chat.

A.2. Testing Environment and Record

The original test plan targeted ordinary development workstations with at least 16 GB of RAM and four CPU cores, running Windows, macOS, or Linux. The backend required Python, FAISS, SPARQL support, PyTest, spaCy, SentenceTransformer, and OpenAI access. The frontend required Node.js, React, and a modern browser. Live DBpedia and OpenAI tests required a stable network connection; local retrieval required the downloaded corpus, embedding array, and FAISS index.

Each commit or pull request to the main branch was intended to trigger GitHub Actions. Unit tests isolated individual handlers, integration tests connected small groups, and functional tests exercised the system manually. Performance tests submitted the same Alan Turing prompt three times and recorded component durations. Client meetings supplied user-acceptance feedback. Figure 5 preserves the original timing screenshots; the accessible values and interpretation limits appear in Table 4.

A.3. Operational and Delivery Record

To run the delivered Docker configuration, a user installs Docker, obtains the frontend and backend images described in the repository, supplies an OpenAI API key, starts Docker Compose, and opens <http://localhost:3000>. For local development, the user installs the Python and Node.js dependencies, downloads the embedding and FAISS artifacts, starts FastAPI on port 8000, and starts the React development server on port 3000. The user then enters a question, waits for retrieval and generation, and reviews the response.

The final report states that code and deployment were complete on April 18, 2025. HackerEarth did not adopt the code into its production system. The artifact was delivered for demonstration and local testing, with no physical equipment. OpenAI credentials were intentionally excluded.

A.4. Project Management

The team held weekly one-hour planning meetings, weekly half-hour client meetings, and four semester meetings with the capstone instructor. Team meetings used a GitHub Kanban board to assign sprint work and track completion. Client meetings supported status updates, demonstrations, and technical feedback. The original planning-board and performance-testing records are reproduced in Figure 5.

A.5. Glossary

CI/CD: continuous integration and continuous deployment. **Knowledge graph:** a graph-structured representation of entities and relations. **LLM:** a large language model trained to process and generate language. **NLP:** natural-language processing. **RAG:** generation conditioned on information retrieved from an external source. **Vector embedding:** a numeric representation used to compare items geometrically. **Vector search:** retrieval based on similarity among vector embeddings. **UI:** the interface through which a user interacts with the system.

A.6. Original Project Records

Figure 5 collects the original test summary, timing screenshots, planning board, and performance-testing issue. These images document the capstone artifact; they introduce no additional trials beyond the values transcribed in the main paper.

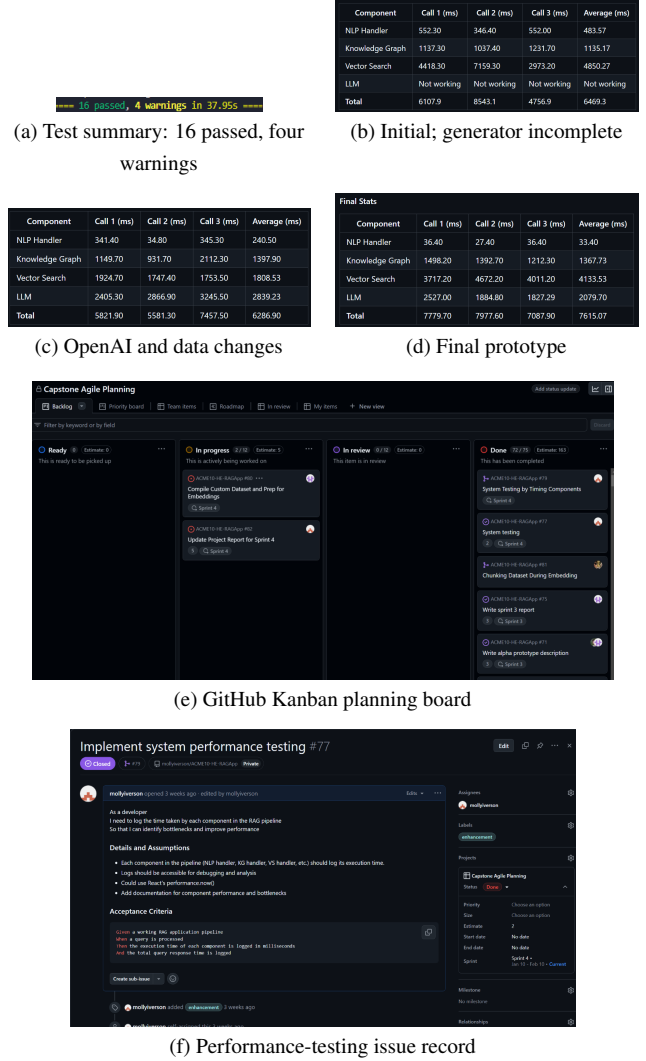


Figure 5. Original testing, timing, and project-management records reproduced from the final report.